APUNTES EXAMEN · ENTORNO CLIENTE

Ventanas · Temporizadores · Eventos · Formularios · DOM

0. CÓMO ATACAR UN EJERCICIO DE EXAMEN

- La profe da el **HTML**: apunta los **id** y los **name** de lo que vayas a usar antes de escribir nada.
- Estructura fija: una función **inicio()** con todos los addEventListener y, al final, **document.addEventListener("DOMContentLoaded", inicio)**.
- Lo que usen varias funciones (temporizadores, ventana hija, arrays) se declara como **var global** arriba del todo.
- **Antes de escribir código: lee el enunciado ENTERO y haz una lista de pasos** (cada frase suele ser un punto). Tacha cada paso al hacerlo; el último paso suele ser el que se olvida (cerrar la ventana, escribir en el padre, redirigir…).
- **Un apartado = una función.** Hazlos en orden y pruébalos uno a uno con F12 (consola).
- Si falla y no ves nada: F12 → Consola → mira el error en rojo (nombre mal escrito, falta un id, variable no declarada…).

```
var temporizador;      // globales: las ven todas las funciones
var ventana;
var datos = [];

function inicio() {
  document.getElementById("boton").addEventListener("click", hacerAlgo);   // SIN paréntesis
  temporizador = setInterval(otraFuncion, 1000);
}

function hacerAlgo() { ... }
function otraFuncion() { ... }

document.addEventListener("DOMContentLoaded", inicio);
```

Receta de temporizadores / ventanas

¿Qué piden?	Qué usar
Hacer algo cada X tiempo	setInterval(funcion, ms) y guardarlo en una var para clearInterval
Hacer algo UNA vez dentro de X tiempo	setTimeout(funcion, ms)
Mostrar la hora / fecha	new Date() DENTRO de la función + getHours(), getMinutes()… + toLocaleTimeString()
Dos dígitos (05 en vez de 5)	if (x < 10) { x = "0" + x; }
Abrir ventana y escribir en ella	v = window.open("a.html","","width=400,height=300"); v.document.write(…)
La hija escribe en el padre	window.opener.document.getElementById("id").innerHTML = …
Cerrar ventana	v.close() / window.close() (ella misma)

1. VENTANAS

```
let v = window.open("segunda.html", "", "width=300,height=200,top=100,left=100");
// 1º: URL ("" = ventana vacía)   2º: nombre   3º: parámetros separados por comas
```

Parámetro	Qué controla
width / height	ancho y alto en píxeles
top / left	posición desde el borde superior / izquierdo de la pantalla
toolbar, location, menubar, status	barra de herramientas, de direcciones, de menú, de estado (yes/no o 1/0)
scrollbars, resizable	barras de scroll / se puede redimensionar

Desde la ventana padre a la hija (v = la variable de la ventana)

```
v.document.write("<h1>Hola</h1>");               // escribir en la hija
v.document.body.innerHTML += "<p>texto</p>";     // añadir contenido
v.document.body.style.backgroundColor = "green"; // cambiar fondo
v.moveBy(200, 100);   v.moveTo(500, 200);        // mover relativo / posición absoluta
v.resizeBy(100, 100); v.resizeTo(400, 200);      // aumentar / tamaño exacto
v.focus();                                       // traer al frente (tras mover/redimensionar)
v.close();                                       // cerrar la hija
if (v.closed) { }                                // true si está cerrada
```

Desde la ventana hija a la que la abrió

```
window.opener.document.body.style.backgroundColor = "red";
window.opener.document.getElementById("id").innerHTML = "texto";
window.opener.variableGlobalDelPadre;            // la hija puede leer las var globales del padre
window.close();                                  // cerrarse a sí misma
window.opener.close();                           // cerrar la padre
```

Dos hijas que se comunican: se guardan las dos en var globales del padre (ventana1, ventana2) y desde una hija se llega a la otra con window.opener.ventana2.document…

Ojo: justo después de window.open la hija aún no ha cargado: si necesitas leer sus elementos, o escribes con v.document.write(…) o dejas que la hija lea del padre cuando cargue (DOMContentLoaded de la hija).

REGLA DE ORO: ¿dónde se ejecuta este código? (principal o hija)

El código corre en…	Su propia página	La otra ventana	Cerrar
La PRINCIPAL (la que hace window.open)	document…	v.document… (v = la ventana que abrió; hay que esperar a que cargue)	v.close() cierra la hija; window.close() cierra la PRINCIPAL
La HIJA (su propio .js o su onclick)	document…	window.opener.document… / window.opener.variable	window.close() cierra la hija; window.opener.close() cierra la principal

window.opener solo existe en la hija (en la principal vale null → error). **Si el enunciado dice «cada página su propio .js»**, haz dos archivos (index.js y segunda.js): así la hija no ejecuta el código de la principal. Si las dos páginas cargan el MISMO .js y hay inicio automático, la hija lo ejecutará también (bucle de popups): protege con if (window.opener == null) o usa onclick en el HTML.

Si solo puedes tocar el JS del padre (no puedes editar el HTML de la hija)

Regla: las variables compartidas (la ventana, el temporizador) van **fuera** de las funciones con var; dentro solo se les asigna (v = window.open(…), SIN let/var). Y el botón de la hija **no existe hasta que la hija carga**, así que hay que esperar:

```
var v;

inicio = () => {
  document.getElementById("abrir").addEventListener("click", abrir);
  document.getElementById("abrir2").addEventListener("click", abrirVacia);
}
```

```
abrir = () => {
  v = window.open("segunda.html", "", "width=300,height=300");
  v.addEventListener("load", () => {                        // esperar a que la hija cargue
    v.document.getElementById("boton").addEventListener("click", comprobar);
  });
}

abrirVacia = () => {
  v = window.open("", "", "width=300,height=300");
  // se crea ya: no hay que esperar
  v.document.write("<input id='texto'><button id='boton'>Comprobar</button>");
  v.document.getElementById("boton").addEventListener("click", comprobar);
}

comprobar = () => {
  let text = v.document.getElementById("texto").value;
  if (text == "") {
    document.getElementById("info").innerHTML = "texto vacio";
  } else {
    v.close();
    document.getElementById("info").innerHTML = "cerrada";
  }
}

window.addEventListener("DOMContentLoaded", inicio);
```

Dos formas: (1) hija con archivo (segunda.html): v.addEventListener("load", () => {…}) y dentro se engancha lo que haga falta de v.document. (2) ventana vacía (""): se rellena con v.document.write(…) y los elementos ya existen, sin esperar. **Nunca** v.document.getElementById(…) justo detrás de window.open con archivo: da null y error.

Plantilla: ventana emergente a los N segundos que escribe en la principal (un .js por página)

js/index.js (solo la principal):

```
inicio = () => {
  setTimeout(abrirVentana, 5000);       // a los 5 segundos se abre la emergente
}

abrirVentana = () => {
  window.open("segunda.html", "segunda", "width=300,height=200");
}

window.addEventListener("DOMContentLoaded", inicio);
```

js/segunda.js (solo la hija):

```
inicio = () => {
  document.getElementById("boton").addEventListener("click", comprobar);
}

comprobar = () => {
  let texto = document.getElementById("texto").value;
  if (texto == "") {
    alert("Escribe algo en el cuadro de texto");
  } else {
    // escribir en la principal
    window.opener.document.getElementById("mensaje").innerHTML = texto;
    window.close();                                                    // cerrar esta ventana
  }
}

window.addEventListener("DOMContentLoaded", inicio);
```

Antes de probar: abrir con Live Server (no doble clic: file:// bloquea window.opener) y **permitir popups** para la página (un window.open que no viene de un clic, como el de un setTimeout, se bloquea).

Plantilla: ruleta con Iniciar / Parar y ventana Resumen 500x500 (todo desde la principal)

```
var vTemp;

const inicio = () => {
  document.getElementById("ini").addEventListener("click", iniciar);
```

```
  document.getElementById("parar").addEventListener("click", parar);
  document.getElementById("resumen").addEventListener("click", ventanaNueva);
};

const iniciar = () => {
  clearInterval(vTemp);                  // evita intervalos duplicados
  vTemp = setInterval(generar, 1000);    // la función del intervalo va APARTE
};

const generar = () => {
  let min = Number(document.getElementById("min").value);
  let max = Number(document.getElementById("max").value);
  let aleatorio = Math.floor(Math.random() * (max - min + 1)) + min;
  document.getElementById("res").value = aleatorio;     // escribir: al .value del elemento
};

const parar = () => {
  clearInterval(vTemp);
};

const ventanaNueva = () => {
  // en la principal: document, no window.opener
  let min = document.getElementById("min").value;
  let v = window.open("resumen.html", "", "width=500,height=500");
  v.addEventListener("load", () => {                    // esperar a que la hija cargue
    v.document.getElementById("resumenSecundaria").innerHTML = min;
    v.document.getElementById("cerrar").addEventListener("click", () => {
      // cerrar la HIJA (window.close() cerraría la principal)
      v.close();
    });
  });
};

window.addEventListener("DOMContentLoaded", inicio);
```

Escribir y leer en un campo: el fallo más repetido

```
let r = document.getElementById("res").value;   // r es una COPIA del texto de ese momento
r = 5;                                          // ✘ NO escribe en el campo
document.getElementById("res").value = 5;       // ✔ escribir en un input (value)
document.getElementById("capa").innerHTML = 5;  // ✔ escribir en un div/p/span (innerHTML)
`${elemento}`                                   // ✘ imprime [object HTMLInputElement]
`${elemento.value}`                             // ✔ el valor
// Temporizador: la función que llamas va declarada APARTE
vTemp = setInterval(generar, 1000);             // y antes: clearInterval(vTemp) para no duplicar
```

Otros objetos del navegador

Objeto	Lo que hay que saber
location	href (URL completa), protocol, host, port, pathname, search (?a=1), hash; location.href = "url" redirige; reload()
history	back(), forward(), go(-1) anterior, go(1) siguiente, length
navigator	userAgent, language, platform, cookieEnabled, onLine
screen	width, height, availWidth, availHeight (sin barra de tareas), colorDepth
window	innerWidth/innerHeight (zona de la página), outerWidth/outerHeight (toda la ventana), opener, closed, scrollBy/scrollTo

2. TEMPORIZADORES

```
let t1 = setTimeout(f1, 5000);   // se ejecuta UNA vez a los 5 s
clearTimeout(t1);                // anula el setTimeout
let t2 = setInterval(f1, 1000);  // se ejecuta CADA 1 s
clearInterval(t2);               // detiene el setInterval
```

Regla: guardar siempre el temporizador en una variable (global si lo para otra función). El tiempo va en milisegundos. La función va SIN paréntesis: setInterval(f1, 1000).

Plantillas

```
// Redirigir tras 5 s
setTimeout(irAGoogle, 5000);
function irAGoogle() { window.location.href = "https://www.google.es"; }

// Cuenta atrás que cierra una ventana
var restantes = 10;
var v = window.open("emergente.html", "", "width=400,height=200");
var intervalo = setInterval(contar, 1000);
function contar() {
  restantes--;
  if (restantes <= 0) {
    clearInterval(intervalo);           // siempre parar el intervalo
    v.close();
  } else {
    v.document.body.innerHTML = `<p>Quedan ${restantes} s</p>`;
  }
}

// Rotar mensajes de un array (uno cada 2 s, al llegar al final vuelve al principio)
var cont = 0;
function mostrar() {
  document.getElementById("resultado").innerHTML = frases[cont];
  cont++;
  if (cont == frases.length) { cont = 0; }
}

// Alternar color/texto cada segundo
function cambiar() {
  let c = document.getElementById("contenedor");
  if (c.style.backgroundColor == "green") { c.style.backgroundColor = "red"; c.innerHTML =
"Sevilla"; }
  else { c.style.backgroundColor = "green"; c.innerHTML = "Betis"; }
}

// Ruleta de números aleatorios + tiempo transcurrido
horaInicio = new Date();                                    // al iniciar
vTemp = setInterval(sortear, 50);
// al parar: restar fechas = milisegundos
segundos = Math.round((new Date() - horaInicio) / 1000);
```

3. DATE, MATH, STRING Y NUMBER

Date

Los **meses empiezan en 0** (enero = 0, diciembre = 11). **getDay()** da 0 = domingo … 6 = sábado.

```
let f  = new Date();                          // ahora
let f2 = new Date(milisegundos);
let f3 = new Date(año, mes, dia, horas, minutos, segundos);
let f4 = new Date(año, mes, dia);             // hora 00:00:00
```

Leer	Devuelve	Cambiar
getDate()	día del mes (1-31)	setDate(día)
getDay()	día de la semana (0-6)	no tiene
getMonth()	mes (0-11)	setMonth(mes)
getFullYear()	año completo	setFullYear(año)
getHours()	hora	setHours(h)
getMinutes()	minutos	setMinutes(m)
getSeconds()	segundos	setSeconds(s)
getTime()	ms desde 1/1/1970	setTime(ms)

Método	Ejemplo de resultado
toString()	Mon Dec 26 2050 08:30:00 GMT…
toDateString()	Mon Dec 26 2050
toLocaleDateString()	26/12/2050
toLocaleString()	26/12/2050 9:30:00
toLocaleTimeString()	9:30:00

```
`${f.getDate()}/${f.getMonth() + 1}/${f.getFullYear()}`         // dd/mm/yyyy (sumar 1 al mes)
// días entre dos fechas (restar da ms)
let dias = (fin - hoy) / (24 * 60 * 60 * 1000);
f.setFullYear(f.getFullYear() - 1);                              // un año atrás
let nombres = ["Domingo","Lunes","Martes","Miércoles","Jueves","Viernes","Sábado"];
nombres[f.getDay()];                                              // día de la semana en español
// ¿es fin de semana?   f.getDay() == 0 || f.getDay() == 6
// año bisiesto: (a % 4 == 0 && a % 100 != 0) || a % 400 == 0
```

Math

Método	Hace
Math.round(x)	entero más cercano
Math.ceil(x) / Math.floor(x)	redondea hacia arriba / hacia abajo
Math.trunc(x)	quita decimales
Math.abs(x)	valor absoluto
Math.max(a,b,c) / Math.min(a,b,c)	mayor / menor
Math.pow(base,exp)	potencia
Math.sqrt(x) / Math.cbrt(x)	raíz cuadrada / cúbica
Math.PI	3.14159…
Math.random()	entre 0 (incluido) y 1 (excluido)

```
Math.floor(Math.random() * 5);                       // 0 a 4
Math.floor(Math.random() * 5) + 6;                   // 6 a 10
Math.floor(Math.random() * (max - min + 1)) + min;   // min a max, ambos incluidos
```

String y Number

El primer índice es 0 y el último length - 1. Las comillas invertidas permiten plantillas: `Hoy es ${dia}`. Escapes: \n salto de línea, \t tabulador, \" comillas dobles, \' simples, \\ barra invertida.

Método String	Hace
length	nº de caracteres (propiedad, sin paréntesis)
charAt(i) o texto[i]	carácter en la posición i
substring(ini, fin)	desde ini hasta fin SIN incluir fin
split(sep)	divide en array
toUpperCase() / toLowerCase()	mayúsculas / minúsculas
trim()	quita espacios al inicio y final
indexOf(x)	primera posición (-1 si no está)
lastIndexOf(x)	última posición
startsWith(x) / endsWith(x)	true o false
replace(a, b)	sustituye solo la primera aparición
includes(x)	true si contiene x

Método Number	Hace	Ejemplo
toFixed(n)	n decimales (devuelve TEXTO)	(12.123).toFixed(2) → "12.12"
isNaN(x)	true si NO es número	isNaN("Hola") → true; isNaN("15") → false
parseInt(x)	texto a entero	parseInt("37hola") → 37; parseInt("hola37") → NaN
parseFloat(x)	texto a decimal	parseFloat("3.14") → 3.14
Number(x)	conversión estricta	Number("37hola") → NaN

```
// iniciales de nombre y apellidos:  datos = "Miguel Polvillo Soto"
let p = datos.split(" ");  p[0].substring(0,1) + p[1].substring(0,1) + p[2].substring(0,1)
// invertir cadena
// o  t.split("").reverse().join("")
let r = ""; for (let i = t.length - 1; i >= 0; i--) { r += t[i]; }
// nº de cifras de un número:  String(n).length
```

Conversión automática (trampas)

Expresión	Resultado
'2' * 3	6 (el * convierte a número)
'2' + 3 / 2 + '3'	"23" (el + concatena si hay texto)
9 + 4 + "10"	"1310" (de izquierda a derecha)
'Hola' * 3	NaN
null → número / undefined → número	0 / NaN
typeof 1 + 1	"number1" · typeof null → "object"
isNaN("") · Number("")	false · 0 → un campo VACÍO pasa isNaN: comprobar antes valor == ""

Validar números (NaN)

```
// Pedir un número hasta que sea válido
let n;
do {
  n = parseInt(prompt("Introduce un número"));
} while (isNaN(n));

// Entero positivo
let n;
do { n = Number(prompt("Introduce un número entero positivo")); } while (isNaN(n) || n <= 0);

// A prueba de todo (vacío y Cancelar)
let texto;
do { texto = prompt("Introduce un número"); } while (texto === null || texto.trim() === "" ||
isNaN(Number(texto)));
let n = Number(texto);
```

prompt devuelve texto (o null si Cancelas) → convertir con Number() / parseInt() para operar. **confirm** devuelve true/false. **alert** no devuelve nada.

4. FUNCIONES, ARRAYS Y OBJETOS

Funciones: las 3 formas

```
function triple(n = 0) { return n * 3; }              // 1ª tradicional (n = 0: valor por
defecto)
let triple = function (n = 0) { return n * 3; };      // 2ª anónima
let triple = (n = 0) => { return n * 3; };            // 3ª flecha
let triple = n => n * 3;                              // flecha de una expresión: return
implícito
```

La anónima y la flecha hay que declararlas **antes** de usarlas. Sin return la función devuelve undefined. triple(2,3) no falla: el argumento que sobra se ignora.

Arrays: métodos

Método	Hace	¿Modifica el original?
a.length	nº de elementos (propiedad)	—
x instanceof Array	true si x es un array	—
push(x) / pop()	añade al final / quita el último	SÍ
unshift(x) / shift()	añade al inicio / quita el primero	SÍ
concat(b)	une dos arrays en uno nuevo	NO
slice(ini, fin)	subarray (fin NO incluido)	NO
splice(ini, nBorrar, nuevos…)	borra y/o inserta	SÍ
join(sep)	array → texto	NO
indexOf(x) / includes(x)	posición (-1 si no está) / true-false	NO
reverse() / sort()	invierte / ordena (como TEXTO)	SÍ
forEach(fn)	recorre: fn(elemento, índice)	NO
filter(fn)	array nuevo con los que cumplen	NO
map(fn)	array nuevo con cada elemento transformado	NO

```
let a = [3, 4, "Hola", [99, 55, 33]];   a[3][1] → 55    a[2][1] → "o"
let c = ["blanco", "negro", "azul", "lila"];
c.splice(2, 0, "verde");        // inserta "verde" en la posición 2 sin borrar
c.splice(3, 1);                 // borra 1 elemento desde la posición 3
[10, 9, 1].sort((a, b) => a - b);                    // orden numérico
"hello".split("").reverse().join("");                // "olleh"

const pares = numeros.filter(n => n % 2 === 0);      // los que cumplen
const dobles = numeros.map(n => n * 2);              // mismo nº, transformados
let suma = 0; numeros.forEach(n => suma += n);       // suma
usuarios.filter(u => u.edad >= 18).map(u => `${u.nombre} (${u.edad} años)`);   // encadenar
```

Recorridos

```
for (let i = 0; i < a.length; i++) { a[i] }     // índice; los huecos salen como undefined
for (let i in a) { a[i] }                          // índice (texto); SE SALTA los huecos
for (let v of a) { v }                             // valor; NO se salta los huecos
delete a[2];                                       // deja hueco, NO cambia length
```

Objetos literales

```
let perro = { nombre: "Scott", edad: 5, ladrar: function () { return `${this.nombre} ladra`; } };
perro.nombre;  perro["nombre"];                    // acceder
perro.tamaño = "Grande";  perro.edad = 8;          // añadir / modificar
delete perro.edad;                                 // eliminar
for (let prop in perro) { if (typeof perro[prop] != "function") console.log(prop, perro[prop]); }
// this = el objeto dueño del método (en métodos usa function(){}, no flecha)
alumnos.push({ nombre: "Ana", edad: 20 });         // array de objetos
alumnos.splice(alumnos.indexOf(alumno), 1);        // quitar un objeto del array
```

5. EVENTOS

Un **evento** es una acción del navegador (clic, tecla, enviar formulario…). El código que responde es el **manejador**.

Grupo	Eventos (manejador = "on" + nombre)
Ratón	click · dblclick · mousedown · mouseup · mouseover (ENTRA el puntero) · mouseout (SALE) · mousemove (se repite dentro)
Teclado	keydown (pulsar; se repite) · keypress (alfanumérica; obsoleto) · keyup (soltar)
Formulario	submit · change (cambia el valor) · focus (gana el foco) · blur (pierde el foco) · reset · select · resize · scroll · error · abort
Carga	load (todo cargado) · unload (se cierra) · DOMContentLoaded (documento cargado)
Arrastre	drag (arrastrando) · dragstart (empieza) · dragend (termina) · dragover (sobre destino) · drop (suelta en destino)

Las 3 formas de manejar eventos

```
// 1ª EN LÍNEA (no recomendada: mezcla HTML y JS)
<button onclick="alertar()">Pulsa</button>

elemento.onclick = hacerAlgo;                          // 2ª TRADICIONAL: SIN paréntesis
elemento.onclick = null;                               //     quitar el manejador
elemento.onclick();                                    //     dispararlo a mano
// Solo admite UN manejador por evento. hacerAlgo() con paréntesis la ejecuta ya (error típico)

elemento.addEventListener("click", funcion, false);    // 3ª MODERNA (el evento SIN "on")
elemento.removeEventListener("click", funcion);        //     quitar (la función debe tener
nombre)
// Permite VARIOS manejadores del mismo evento. this = el elemento que provocó el evento
```

El objeto event

```
const alertar = (e) => {            // e = event (puede llamarse como quieras)
  alert(e.type);                      // "click"
  alert(e.target);                    // elemento que disparó el evento
  e.target.style.color = "blue";
  // anula la acción por defecto (enviar formulario, seguir un enlace)
  e.preventDefault();
};
document.addEventListener("keydown", (e) => console.log(e.key));
```

Plantillas

```
input.addEventListener("focus", () => input.style.backgroundColor = "yellow");
input.addEventListener("blur", () => { input.value = input.value.toUpperCase(); });
boton.addEventListener("mouseover", () => boton.style.border = "3px solid green");
boton.addEventListener("mouseout", () => boton.style.border = "3px solid orange");
// eventos con un bucle (let i: cada vuelta su i)
for (let i = 0; i <= 9; i++) {
  document.getElementById("b" + i).addEventListener("click", () => { resultado.innerHTML = i; });
}
// contador: n++ / n--  y  innerHTML = n + (n == 3 ? " Alcanzado valor 3" : "")
```

Arrastrar y soltar

```
<div id="origen" draggable="true">Arrástrame</div>   <div id="destino">Suelta aquí</div>
origen.addEventListener("dragstart", e => e.dataTransfer.setData("id", e.target.id));
// imprescindible para poder soltar
destino.addEventListener("dragover", e => e.preventDefault());
destino.addEventListener("drop", e => {
  e.preventDefault();
  destino.appendChild(document.getElementById(e.dataTransfer.getData("id")));
});
```

6. FORMULARIOS

Seleccionar el formulario y sus controles

Forma	Ejemplo
por id (la mejor)	document.getElementById("form")
por name	document.forms["nombre"]
por posición	document.forms[0]
por etiqueta	document.getElementsByTagName("form")[0] ← devuelve COLECCIÓN: poner [0]
un control	form.elements["nombre"] · form.elements[1] · document.forms["f"].cEntrada · document.getElementById("id")

```
form.elements.length;  form.action;  form.method;       // propiedades (se pueden asignar)
form.submit();  form.reset();                           // enviar / vaciar desde JS
for (let i = 0; i < form.elements.length; i++) {        // recorrer todos los controles
  if (form.elements[i].type == "text") { form.elements[i].value = ""; }
}
```

Input text, checkbox, radio y select

Control	Propiedades y métodos
text	value (SIEMPRE texto → Number()) · defaultValue · placeholder · disabled · readOnly · maxLength · size · type · name · select() focus() blur()
checkbox	checked (true/false, se puede asignar) · value · name · disabled · focus() blur()
radio	mismo name = grupo (solo uno marcado) · checked · value · se recorre el grupo con getElementsByName
select	value · selectedIndex (empieza en 0) · options (cada una: text, value, selected) · length · multiple

```
// CHECKBOX con name común: sumar los marcados
let checks = document.getElementsByName("num"); let suma = 0;
for (let i = 0; i < checks.length; i++) {
  if (checks[i].checked) { suma += Number(checks[i].value); }
  checks[i].checked = false;                            // desmarcar
}
// RADIO: cuál está marcado
let radios = document.getElementsByName("grupo"), sel = "";
for (let i = 0; i < radios.length; i++) { if (radios[i].checked) { sel = radios[i].value; } }
// SELECT simple
select.selectedIndex;  select.options[select.selectedIndex].text;  select.value;
select.options[2].selected = true;                      // seleccionar la de índice 2
// SELECT múltiple: textos de las seleccionadas
let elegidas = [];
for (let i = 0; i < select.options.length; i++) { if (select.options[i].selected)
elegidas.push(select.options[i].text); }
alert(elegidas.join(", "));
```

Validar un formulario (solo el primer error)

```
formulario.addEventListener("submit", validar);
function validar(e) {
  e.preventDefault();                                   // no enviar todavía
  let nombre = document.getElementById("nombre").value.trim();
  let edad = document.getElementById("edad").value.trim();
  let res = document.getElementById("resultado");  res.style.color = "red";
  if (nombre == "") { res.innerHTML = "El nombre es obligatorio"; }
  else if (edad == "" || isNaN(edad) || edad < 0 || edad > 105) { res.innerHTML = "Edad entre 0 y
105"; }
  else if (provincia.value == "0") { res.innerHTML = "Elige provincia"; }
  else if (confirm("¿Enviar?")) { formulario.submit(); }  // enviar a mano
}
```

Contenido, atributos y estilos

```
el.innerHTML = "<b>Hola</b>";   el.textContent = "<b>Hola</b>";    // interpreta HTML / texto
puro
img.src = "nueva.png";   img.setAttribute("src", "nueva.png");
// en JS los guiones pasan a camelCase (background-color → backgroundColor)
el.style.backgroundColor = "red";
el.className = "clase";   el.setAttribute("class", "clase");
```

7. DOM

El DOM es el árbol de nodos de la página: elementos, atributos, texto, comentarios y el nodo raíz document.

Acceder a los elementos

```
document.getElementById("id")                    // UN elemento
document.getElementsByTagName("p")[0]            // COLECCIÓN → indexar con [0]
document.getElementsByName("grupo")              // colección
document.getElementsByClassName("fila")          // colección
document.forms   document.images   document.links   document.body
```

Moverse por el árbol

Nodos (incluye texto y comentarios)	Solo elementos
childNodes	children
firstChild	firstElementChild
lastChild	lastElementChild
parentNode	parentElement
nextSibling	nextElementSibling
previousSibling	previousElementSibling

Cuidado: los espacios y saltos de línea del HTML cuentan como nodos de texto (por eso childNodes tiene más nodos que children). Casi siempre usa las versiones **Element**.

Crear, insertar, borrar y reemplazar

```
let p = document.createElement("p");                // 1. crear (la etiqueta sin <>)
p.innerHTML = "Soy el nuevo";   p.id = "p3";        // 2. rellenar (o createTextNode +
appendChild)
div.appendChild(p);                                 // 3. COLGARLO del árbol (sin esto no se ve)
div.insertBefore(p, div.firstElementChild);         // antes de un hijo (p4 delante del primero)
div.removeChild(hijo);   hijo.remove();             // borrar (con el padre / directamente)
div.replaceChild(nuevo, antiguo);                   // reemplazar
// de un botón/checkbox → td → tr: borrar la fila
nodo.parentNode.parentNode.remove();
```

Plantillas

```
// lista desde un array con createElement
let h2 = document.createElement("h2"); h2.innerHTML = "Estaciones";
document.body.appendChild(h2);
let ul = document.createElement("ul");
for (let i = 0; i < estaciones.length; i++) {
  let li = document.createElement("li"); li.innerHTML = estaciones[i]; ul.appendChild(li);
}
document.body.appendChild(ul);
// con innerHTML
let html = ""; for (let c of continentes) { html += `<li>${c}</li>`; }  ul.innerHTML = html;

// tabla: fila (tr) + columnas (td) dentro de tbody
let tr = document.createElement("tr"); tr.id = obj.id;
let td = document.createElement("td"); td.innerHTML = obj.nombre; tr.appendChild(td);
document.getElementById("cuerpo").appendChild(tr);
// checkbox en una celda
let ch = document.createElement("input"); ch.type = "checkbox"; ch.name = "marcar"; ch.id =
"checkbox1";
// borrar filas marcadas: recorrer AL REVÉS porque la colección se acorta al borrar
for (let i = checks.length - 1; i >= 0; i--) { if (checks[i].checked)
{ checks[i].parentNode.parentNode.remove(); } }
// elemento sin id: segundo párrafo → document.getElementsByTagName("p")[1].style.backgroundColor
= "red";
// ocultar / mostrar: el.style.display = "none"  o  "block"
```

nodeValue, childNodes vs children, atributos

```
// <div id="caja">  <p>Hola</p>  <p>Adiós</p>  </div>
let caja = document.getElementById("caja");
caja.childNodes.length          // 5  (texto, p, texto, p, texto: los espacios cuentan)
caja.children.length            // 2  (solo los <p>)
caja.firstChild                 // nodo de texto (espacio)      caja.firstElementChild  → el primer <p>
caja.firstElementChild.firstChild.nodeValue     // "Hola"  (el texto de un elemento es su primer hijo)
caja.firstElementChild.nodeName                 // "P"     nodeType: 1 elemento, 3 texto
caja.firstElementChild.firstChild.nodeValue = "Nuevo";     // cambiar el texto (= innerHTML)
caja.children[1].previousElementSibling         // el primer <p>
// atributos
let enlace = document.getElementsByTagName("a")[0];
enlace.getAttribute("href");   enlace.setAttribute("href", "https://www.google.es");
enlace.removeAttribute("href");
enlace.href = "https://www.google.es";   enlace.style.color = "red";   enlace.style.fontSize = "20px";
document.getElementsByTagName("p")[0].setAttribute("class", "parrafo1");   // = className = "parrafo1"
document.getElementsByTagName("p")[0].classList.add("rojo");   classList.remove("rojo");
classList.toggle("rojo");
let filas = document.getElementsByClassName("fila");           // colección
for (let i = 0; i < filas.length; i++) { filas[i].style.backgroundColor = "yellow"; }
```

Crear un formulario entero desde JS (atributos, estilos, placeholder)

```
const inicio = () => {
  document.getElementById("boton").addEventListener("click", crearFormulario);
};

const crearFormulario = () => {
  const seccion = document.getElementById("ContentFormulario");

  const form = document.createElement("form");            // 1. crear el formulario
  form.style.width = "300px";                              //    estilos con style.propiedad
  form.setAttribute("action", "https://www.google.es");    //    atributos con setAttribute
  form.setAttribute("method", "get");

  const campos = ["Nombres", "Apellidos", "Email", "Asunto", "Mensaje"];
  for (let i = 0; i < campos.length; i++) {
    const input = document.createElement("input");         // 2. un input por campo
    input.setAttribute("type", "text");
    input.setAttribute("placeholder", campos[i]);
    if (campos[i] == "Mensaje") {
      // cssText: varios estilos de golpe
      input.style.cssText = "width:100%;height:200px;margin:10px 0px;padding:5px";
    } else {
      input.style.cssText = "width:100%;margin:10px 0px;padding:5px";
    }
    form.appendChild(input);
  }
```

```
  const enviar = document.createElement("input");          // 3. botón Enviar
  enviar.setAttribute("type", "button");
  enviar.setAttribute("value", "Enviar");
  enviar.style.cssText = "width:100px;margin:10px
0px;padding:10px;background:#F05133;color:#fff;border:solid 1px #000";
  enviar.addEventListener("click", () => { alert("Formulario enviado"); });
  form.appendChild(enviar);

  seccion.appendChild(form);                               // 4. colgar el formulario de la sección
};

window.addEventListener("DOMContentLoaded", inicio);
```

Leer un input type="date": dd/mm/aaaa y años bisiestos

```
const inicio = () => {
  document.getElementById("boton").addEventListener("click", mostrar);
};

const mostrar = () => {
  const fecha = document.getElementById("fecha").value;       // viene como "2000-05-17" (aaaa-mm-dd)
  const partes = fecha.split("-");                            // ["2000", "05", "17"]
  const formato = `${partes[2]}/${partes[1]}/${partes[0]}`;   // dd/mm/aaaa

  let bisiestos = 0;
  const actual = new Date().getFullYear();
  for (let a = Number(partes[0]); a <= actual; a++) {
    if ((a % 4 == 0 && a % 100 != 0) || a % 400 == 0) {       // regla del año bisiesto
      bisiestos++;
    }
  }
  document.getElementById("resultado").innerHTML = `Nacimiento: ${formato}. Años bisiestos desde
entonces: ${bisiestos}`;
};

window.addEventListener("DOMContentLoaded", inicio);
```

Mostrar / ocultar con un enlace (varios párrafos: id con número)

```
const inicio = () => {
  for (let i = 1; i <= 3; i++) {                              // let i: cada vuelta conserva su i
    document.getElementById("enlace_" + i).addEventListener("click", (e) => {
      e.preventDefault();                                     // el enlace no salta a #
      alternar(i);
    });
  }
};

const alternar = (i) => {
  const parrafo = document.getElementById("contenidos_" + i);
  const enlace = document.getElementById("enlace_" + i);
  if (parrafo.style.display == "none") {
    parrafo.style.display = "block";
    enlace.innerHTML = "Ocultar Contenido";
  } else {
    parrafo.style.display = "none";
    enlace.innerHTML = "Mostrar Contenido";
  }
};

window.addEventListener("DOMContentLoaded", inicio);
```

Cambiar el texto de un <li> por posición (prompt)

```
const inicio = () => {
  document.getElementById("boton").addEventListener("click", cambiar);
};

const cambiar = () => {
  const texto = prompt("Texto a introducir");
  const pos = parseInt(prompt("Posición (1 a 10)"));
  // colección: se indexa con [ ]
  const items = document.getElementById("lista").getElementsByTagName("li");
  if (isNaN(pos) || pos < 1 || pos > items.length) {
    alert("Posición no válida");
  } else {
    items[pos - 1].innerHTML = texto;                    // posición 1 = índice 0
  }
};

window.addEventListener("DOMContentLoaded", inicio);
```

Tabla creada con dos for anidados y estilos distintos por celda

```
const inicio = () => {
  const tabla = document.createElement("table");
  tabla.setAttribute("border", "1");
  for (let i = 1; i <= 2; i++) {
    const tr = document.createElement("tr");
    for (let j = 1; j <= 2; j++) {
      const td = document.createElement("td");
      td.innerHTML = `Posición: ${i}${j}`;
      if (j == 1) {
        td.style.backgroundColor = "red";
      } else {
        td.style.backgroundColor = "yellow";
        td.style.fontWeight = "bold";
      }
      tr.appendChild(td);
    }
    tabla.appendChild(tr);
  }
  document.body.appendChild(tabla);
};

window.addEventListener("DOMContentLoaded", inicio);
```

Tabla de objetos: filas, checkbox, seleccionar todos, eliminar, añadir, hover

```
const productos = [
  { id: "p1", nombre: "Teclado", precio: 25, codigo: "A01" },
  { id: "p2", nombre: "Ratón", precio: 12, codigo: "A02" },
  { id: "p3", nombre: "Monitor", precio: 150, codigo: "B01" },
  { id: "p4", nombre: "Webcam", precio: 40, codigo: "B02" }
];
var hayColumna = false;   // ¿ya se añadió la columna de checkbox?
var numCheck = 0;         // para los ids: checkbox1, checkbox2...

const inicio = () => {
  for (let i = 0; i < productos.length; i++) {
    crearFila(productos[i]);
  }
  document.getElementById("btnColumna").addEventListener("click", anadirColumna);
  document.getElementById("btnTodos").addEventListener("click", () => marcarTodos(true));
  document.getElementById("btnNinguno").addEventListener("click", () => marcarTodos(false));
  document.getElementById("btnBorrar").addEventListener("click", eliminarMarcadas);
  document.getElementById("add").addEventListener("click", anadirProducto);
};

// 17: crear una fila (tr) con sus columnas (td) y colgarla del tbody
const crearFila = (p) => {
  const tr = document.createElement("tr");
  tr.id = p.id;                                          // el id del objeto es el id de la fila

  const td1 = document.createElement("td");
  const enlace = document.createElement("a");            // 22: el producto es un enlace
  enlace.href = "#";
  enlace.innerHTML = p.nombre;
  td1.appendChild(enlace);
```

```
  const td2 = document.createElement("td");
  td2.innerHTML = p.precio;
  const td3 = document.createElement("td");
  td3.innerHTML = p.codigo;

  tr.appendChild(td1);
  tr.appendChild(td2);
  tr.appendChild(td3);

  // 21: fondo amarillo al pasar por la fila
  tr.addEventListener("mouseover", () => { tr.style.backgroundColor = "yellow"; });
  tr.addEventListener("mouseout", () => { tr.style.backgroundColor = ""; });

  if (hayColumna) {
    crearCheckbox(tr);                                  // las filas nuevas también llevan checkbox
  }
  document.getElementById("cuerpo").appendChild(tr);
};

// 18: columna con checkbox
const crearCheckbox = (tr) => {
  numCheck++;
  const td = document.createElement("td");
  const check = document.createElement("input");
  check.type = "checkbox";
  check.id = "checkbox" + numCheck;
  check.name = "marcar";
  td.appendChild(check);
  tr.appendChild(td);
};

const anadirColumna = () => {
  if (hayColumna) {
    return;                                             // solo una vez
  }
  hayColumna = true;
  const filas = document.getElementById("cuerpo").getElementsByTagName("tr");
  for (let i = 0; i < filas.length; i++) {
    crearCheckbox(filas[i]);
  }
  const th = document.createElement("th");              // cabecera de la nueva columna
  th.innerHTML = "Marcar";
  document.getElementsByTagName("tr")[0].appendChild(th);
};

// 19: seleccionar / deseleccionar todos y eliminar los marcados
const marcarTodos = (valor) => {
  const checks = document.getElementsByName("marcar");
  for (let i = 0; i < checks.length; i++) {
    checks[i].checked = valor;
  }
};

const eliminarMarcadas = () => {
  const checks = document.getElementsByName("marcar");
  for (let i = checks.length - 1; i >= 0; i--) {        // al revés: la lista se acorta al borrar
    if (checks[i].checked) {
      checks[i].parentNode.parentNode.remove();         // checkbox → td → tr: se borra la fila
    }
  }
};

// 20: añadir un producto nuevo desde los inputs
const anadirProducto = () => {
  const nuevo = {
    id: "p" + Date.now(),
    nombre: document.getElementById("nombreN").value,
    precio: document.getElementById("precioN").value,
    codigo: document.getElementById("codigoN").value
  };
  crearFila(nuevo);
};

window.addEventListener("DOMContentLoaded", inicio);
```

Temporizador con Iniciar / Pausar / Reiniciar (sin duplicar intervalos)

```
var intervalo = null;     // null = parado
var inicial;              // valor con el que empieza
```

```
const inicio = () => {
  inicial = Number(document.getElementById("temporizador").innerHTML);
  document.getElementById("iniciar").addEventListener("click", iniciar);
  document.getElementById("pausar").addEventListener("click", pausar);
  document.getElementById("reiniciar").addEventListener("click", reiniciar);
};

const iniciar = () => {
  const valor = Number(document.getElementById("temporizador").innerHTML);
  if (intervalo == null && valor > 0) {                    // evita crear dos intervalos a la vez
    intervalo = setInterval(cuenta, 1000);
  }
};

const cuenta = () => {
  const campo = document.getElementById("temporizador");
  const n = Number(campo.innerHTML) - 1;
  campo.innerHTML = n;
  if (n <= 0) {
    pausar();
    campo.innerHTML = "¡Tiempo agotado!";
  }
};

const pausar = () => {
  clearInterval(intervalo);
  intervalo = null;
};

const reiniciar = () => {
  pausar();
  document.getElementById("temporizador").innerHTML = inicial;
};

window.addEventListener("DOMContentLoaded", inicio);
```

8. EJERCICIO TIPO EXAMEN RESUELTO (todo el temario)

Enunciado. Dada la página index.html (gestión de alumnos), programa el JavaScript (js/index.js) para que:

- **1. (Temporizadores, Date, función flecha)** El cuadro #reloj muestra, en formato HH:MM:SS, el tiempo que queda hasta que acabe el día y se actualiza cada segundo. El botón «Parar reloj» lo detiene y pone el botón en rojo.
- **2. (Eventos)** Al perder el foco, el nombre pasa a mayúsculas. Al pasar el ratón por una fila de la tabla se pone amarilla y al salir vuelve a su color.
- **3. (Formularios)** Al enviar el formulario no se recarga la página y se valida: nombre no vacío, edad número entre 0 y 105, curso elegido, modalidad marcada y al menos una asignatura. Solo se muestra el primer error, en rojo, en #mensaje.
- **4. (DOM + arrays)** Si todo es correcto, se guarda el alumno (objeto) en un array global, se añade una fila a la tabla con sus datos (asignaturas separadas por coma) y un botón «Borrar» que elimina la fila y el alumno del array. Se muestra «Alumno añadido» en verde, se vacía el formulario y el foco vuelve al nombre.
- **5. (Ventanas)** El botón «Resumen» abre resumen.html (400x300). Esa ventana escribe en #capa el nº de alumnos, cuántos son mayores de edad y sus nombres separados por coma (lee el array del padre).
- **6. (Temporizador + ventanas)** A los 5 segundos de abrirse, la ventana se cierra sola y en #aviso (de la ventana principal) se escribe en azul «Resumen consultado a las HH:MM:SS».

HTML dado: index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Gestión de alumnos</title>
  <script src="js/index.js"></script>
</head>
<body>
  <h2>Gestión de alumnos</h2>

  <div>Tiempo que queda de día:
    <input type="text" id="reloj" readonly>
    <button type="button" id="btnParar">Parar reloj</button>
  </div>

  <form id="formulario">
    Nombre: <input type="text" id="nombre"><br>
    Edad: <input type="text" id="edad"><br>
    Curso:
    <select id="curso">
      <option value="0">Elige curso</option>
      <option value="DAW1">DAW1</option>
      <option value="DAW2">DAW2</option>
    </select><br>
    Modalidad:
    <input type="radio" name="modalidad" value="Presencial"> Presencial
    <input type="radio" name="modalidad" value="Online"> Online<br>
    Asignaturas:
    <input type="checkbox" name="asig" value="Cliente"> Cliente
    <input type="checkbox" name="asig" value="Servidor"> Servidor
    <input type="checkbox" name="asig" value="Java"> Java<br>
    <button type="submit" id="btnAnadir">Añadir alumno</button>
  </form>

  <div id="mensaje"></div>

  <table border="1">
    <thead>
      <tr><th>Nombre</th><th>Edad</th><th>Curso</th><th>Modalidad</th><th>Asignaturas</th><th></th></
tr>
    </thead>
    <tbody id="cuerpo"></tbody>
  </table>

  <button type="button" id="btnResumen">Resumen</button>
  <div id="aviso"></div>
</body>
</html>
```

HTML dado: resumen.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Resumen</title>
  <script src="js/resumen.js"></script>
</head>
<body>
  <h3>Resumen</h3>
  <div id="capa"></div>
</body>
</html>
```

Solución: js/index.js

```
var alumnos = [];        // array global: cada alumno es un objeto literal
var vReloj;              // temporizador del reloj (global para poder pararlo)
var ventana;             // ventana hija (global)

const inicio = () => {
  actualizarReloj();                                          // 1. se ve ya, sin esperar 1 s
  vReloj = setInterval(actualizarReloj, 1000);
  document.getElementById("btnParar").addEventListener("click", pararReloj);

  document.getElementById("nombre").addEventListener("blur", mayusculas);           // 2. eventos
  document.getElementById("formulario").addEventListener("submit", validar);        // 3. formulario
  document.getElementById("btnResumen").addEventListener("click", abrirResumen);    // 5. ventanas
}

// ---------- 1. TEMPORIZADORES + DATE + FUNCIÓN FLECHA ----------
const actualizarReloj = () => {
  const ahora = new Date();                                   // se lee dentro: cada segundo
  let h = 23 - ahora.getHours();
  let m = 59 - ahora.getMinutes();
  let s = 59 - ahora.getSeconds();
  if (h < 10) { h = "0" + h; }                                // dos dígitos
  if (m < 10) { m = "0" + m; }
  if (s < 10) { s = "0" + s; }
  document.getElementById("reloj").value = `${h}:${m}:${s}`;
};

const pararReloj = () => {
  clearInterval(vReloj);
  document.getElementById("btnParar").style.backgroundColor = "red";
}

// ---------- 2. EVENTOS ----------
const mayusculas = () => {
  let n = document.getElementById("nombre");
  n.value = n.value.toUpperCase();
}

// ---------- 3. FORMULARIOS: VALIDAR (solo el primer error) ----------
const validar = (e) => {
  e.preventDefault();                                         // evita que el form recargue la
página
  let nombre = document.getElementById("nombre").value.trim();
  let edad = document.getElementById("edad").value.trim();
  let curso = document.getElementById("curso").value;
  let mensaje = document.getElementById("mensaje");

  // radio: recorrer el grupo y ver cuál está marcado
  let modalidad = "";
  let radios = document.getElementsByName("modalidad");
  for (let i = 0; i < radios.length; i++) {
    if (radios[i].checked) {
      modalidad = radios[i].value;
    }
  }

  // checkbox: guardar los marcados en un array
  let asignaturas = [];
  let checks = document.getElementsByName("asig");
  for (let i = 0; i < checks.length; i++) {
    if (checks[i].checked) {
      asignaturas.push(checks[i].value);
    }
```

```
  }

  mensaje.style.color = "red";
  if (nombre == "") {
    mensaje.innerHTML = "El nombre es obligatorio";
  } else if (edad == "" || isNaN(edad) || edad < 0 || edad > 105) {
    mensaje.innerHTML = "La edad debe ser un número entre 0 y 105";
  } else if (curso == "0") {
    mensaje.innerHTML = "Elige un curso";
  } else if (modalidad == "") {
    mensaje.innerHTML = "Elige una modalidad";
  } else if (asignaturas.length == 0) {
    mensaje.innerHTML = "Marca al menos una asignatura";
  } else {
    // todo correcto
    let alumno = { nombre: nombre, edad: Number(edad), curso: curso, modalidad: modalidad, asignaturas:
asignaturas };
    alumnos.push(alumno);
    crearFila(alumno);                                          // 4. DOM
    mensaje.style.color = "green";
    mensaje.innerHTML = "Alumno añadido";
    document.getElementById("formulario").reset();              // vacía el formulario
    document.getElementById("nombre").focus();                  // y deja el foco en el nombre
  }
}

// ---------- 4. DOM: CREAR FILA Y BORRARLA ----------
const crearFila = (alumno) => {
  let tr = document.createElement("tr");                        // 1. crear

  let datos = [alumno.nombre, alumno.edad, alumno.curso, alumno.modalidad, alumno.asignaturas.join(",
")];
  for (let i = 0; i < datos.length; i++) {
    let td = document.createElement("td");
    td.innerHTML = datos[i];                                    // 2. rellenar
    tr.appendChild(td);                                         // 3. colgar
  }

  let tdBorrar = document.createElement("td");
  let boton = document.createElement("button");
  boton.type = "button";
  boton.innerHTML = "Borrar";
  boton.addEventListener("click", function () {
    tr.remove();                                                // quita la fila de la página
    alumnos.splice(alumnos.indexOf(alumno), 1);                 // y el alumno del array
  });
  tdBorrar.appendChild(boton);
  tr.appendChild(tdBorrar);

  // 2. EVENTOS: resaltar la fila al pasar el ratón
  tr.addEventListener("mouseover", function () { tr.style.backgroundColor = "yellow"; });
  tr.addEventListener("mouseout", function () { tr.style.backgroundColor = ""; });

  document.getElementById("cuerpo").appendChild(tr);
}

// ---------- 5. VENTANAS + 6. TEMPORIZADOR ----------
const abrirResumen = () => {
  ventana = window.open("resumen.html", "", "width=400,height=300");
  setTimeout(cerrarResumen, 5000);                              // a los 5 segundos
}

const cerrarResumen = () => {
  ventana.close();
  let aviso = document.getElementById("aviso");
  aviso.innerHTML = "Resumen consultado a las " + new Date().toLocaleTimeString();
  aviso.style.color = "blue";
}

window.addEventListener("DOMContentLoaded", inicio);
```

Solución: js/resumen.js

```
// Ventana hija: lee los datos de la ventana padre con window.opener
const inicio = () => {
  let padre = window.opener;
  let lista = padre.alumnos;                                    // el array global del padre

  let nombres = lista.map(a => a.nombre).join(", ");            // map + join
  let mayores = lista.filter(a => a.edad >= 18).length;         // filter

  document.getElementById("capa").innerHTML = `
    <p>Nº de alumnos: ${lista.length}</p>
    <p>Mayores de edad: ${mayores}</p>
    <p>Nombres: ${nombres}</p>`;
}

window.addEventListener("DOMContentLoaded", inicio);
```

9. ERRORES TÍPICOS (revisar antes de entregar)

- **Paréntesis** en el manejador: addEventListener("click", f) ✔️ · addEventListener("click", f()) ✘ (se ejecuta al cargar).
- **Variable local** que otra función necesita (el temporizador, la ventana): declararla global (var) fuera de las funciones.
- **Elemento creado y no colgado:** falta appendChild / insertBefore.
- **getElementsBy... devuelve colección:** [0] o recorrerla (getElementById es el único que devuelve uno solo).
- **value es texto:** "5" + 1 = "51". Convertir con Number() / parseInt().
- **Formulario que recarga** al validar: e.preventDefault() (o el botón type="button").
- **El script va antes del body:** usar DOMContentLoaded para que los elementos ya existan.
- **Borrar mientras se recorre:** recorrer al revés (i--).
- **new Date() fuera de la función** del temporizador: la hora se queda congelada.
- **Meses empiezan en 0;** getDay(): domingo = 0.
- **clearInterval/clearTimeout** necesitan la variable donde se guardó el temporizador.
- **Dónde corre el código:** en la hija usa window.opener para llegar al padre (en la principal vale null → error); en la principal usa v. Justo tras window.open la hija aún no ha cargado: esperar "load" o escribir con v.document.write.
- **window.close() en código de la principal cierra la principal:** para cerrar la hija, v.close().
- **Popups:** el navegador bloquea window.open que no viene de un clic (un setTimeout, DOMContentLoaded): permitir popups y abrir con Live Server (no doble clic).
- **Leer un input dentro del clic**, no en inicio (al cargar está vacío).
- **Escribir en un campo:** elemento.value = x (no `let r = el.value; r = x`). Imprimir el valor: ${el.value}, no ${el}.
- **Campo vacío:** isNaN("") es false y Number("") es 0 → comprobar valor == "" antes de validar el número.
- **el.style.backgroundColor solo lee estilos puestos por JS o inline** (vale "" si viene de un <style>): para alternar estados usa un contador (cont % 2) o una variable.
- **Dos if seguidos se anulan** (el segundo ve lo que cambió el primero): usar else if.
- **Una función const/flecha se declara ANTES de usarla** (setInterval(f1) con f1 declarada después da error): declarar la función del temporizador aparte.
- **Antes de iniciar un setInterval:** clearInterval(vTemp), o Iniciar dos veces crea dos temporizadores.
- **Mismo name en radios** (si no, se pueden marcar varios); checkbox con name común + getElementsByName.
- **Comparar con ===** y recordar que `"2" == 2` es true.